invariant: every element is not greater than its successor. The purpose of invariants is making sure the conditions necessary to live up to the contract with the external clients are always met. They are the internal counterpart of the guarantees in the API.

3. A **data model** to host the data. This can be a raw chunk of memory, a list, a tree etc…
4. **Algorithms**: the internal logic that is used to update the data structure while making sure that the invariants are not violated.

Table 2.1 API and contract for priority queue

Abstract Data Structure: Priority Queue	

API	`class PriorityQueue {`
	`  top() → element`
	`  peek() → element`
	`  insert(element, priority)`
	`  remove(element)`
	`  update(element, newPriority)`
	`}`

Contract with client	The top element returned by the queue is always the element with highest priority currently stored in the queue

In appendix C we also clarify how there is a difference **between abstract data structures** and **concrete data structures**. The former includes points **(a)** and **(b)** above, describing at a high level how clients will interact with it and the results and performance of operations. The latter builds on the principles and API expressed by the abstract